SRI CHANDRASEKHARENDRA SARASWATHI VISWA MAHAVIDYALAYA

(UNIVERSITY ESTABLISHED UNDER SECTION 3 OF UGC ACT 1956)

ENATHUR, KANCHIPURAM – 631 561

DEPARTMENT OF COMPUTER SCIENCE & ENGINEERING

Name : S.SAI SRI PRIYA

Reg. No : 11249A338

Class : II B.E. (CSE)

Section : S4

Course Code: BCSF244AI2

Course Name: Data Science Toolkit Lab.

SRI CHANDRASEKHARENDRA SARASWATHI VISWA MAHAVIDHYALAYA

(UNIVERSITY ESTABLISHED UNDER SECTION 3 OF UGC ACT 1956)
ENATHUR, KANCHIPURAM – 631 561

DEPARTMENT OF COMPUTER SCIENCE & ENGINEERING

BONAFIDE CERTIFICATE

This is to certify that this is the bonafide record of work done by **Mr/Ms. S.SAI SRI PRIYA** ,

with **Reg. No 11249A338** of **II-B.E.(CSE)** during the academic year 2025 – 2026.

Station: Enathur.

Date: 22-04-2026.

Staff-in-charge **Head of the Department**

Submitted for the Practical examination held on_____.

Examiner-1 **Examiner-2**

INDEX

SL.N0	Name of the experiment	Date	Page no	Signature
01.	NUMPY, PANDAS, MATPLOTLIB, SCIKIT-LEARN PROGRAMS	02-02-2026		
02.	PERFORM DATA EXPLORATION AND PREPROCESSING IN PYTHON.	14-02-2026		
03.	IMPLEMENT REGULARIZED LINEAR REGRESSION.	14-02-2026		
04.	IMPLEMENT NAÏVE BAYES CLASSIFICATION.	23-02-2026		
05.	IMPLEMENT REGULARIZED LOGISTIC REGRESSION.	02-03-2026		
06.	BUILD MODELS USING DIFFERENT ENSEMBLING TECHNIQUES.	09-03-2026		
07.	BUILD MODELS USING DECISION TREES.	16-03-2026		
08.	SVM WITH DIFFERENT KERNELS.	16-03-2026		
09.	IMPLEMENT KNN CLASSIFICATION.	23-03-2026		
10.	K-MEANS CLUSTERING WITH PCA AND ELBOW METHOD	23-03-2026		

NUMPY,PANDAS,MATPLOTLIB,SCIKIT-LEARN PROGRAMS

EXP NO: 01
DATE: 02-02-2026

AIM: To Study and Visualize data using numpy,pandas,matplotlib and scikitlearn in python

11249A338 exp 1.ipynb

File Edit View Insert Runtime Tools Help

Commands + Code + Text Run all

[1]

```python
import pandas as pd
data={
    'name':['alice','bob','charlie'],
    'age':[25,30,35],
    'city':['new york','paris','london']
}
df=pd.DataFrame(data)
print(df.head())
print(df.info())
print(df.describe())
ages=df['age']
above_25=df[df['age']>25]
row_0=df.iloc[0]
specific_val=df.loc[0,'city']
print(above_25)
print(row_0)
print(specific_val)
print(df.isnull().sum())
df_clean =df.dropna()
df_filled=df.fillna(0)
print(df_clean)
print(df_filled)
```

```
      name  age      city
0    alice   25  new york
1      bob   30     paris
2  charlie   35    london
<class 'pandas.core.frame.DataFrame'>
```

11249A338 exp 1.ipynb

File Edit View Insert Runtime Tools Help

Commands + Code + Text Run all

```
      name  age      city
0    alice   25  new york
1      bob   30     paris
2  charlie   35    london
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 3 entries, 0 to 2
Data columns (total 3 columns):
 #   Column  Non-Null Count  Dtype
---  ------  --------------  -----
 0   name    3 non-null      object
 1   age     3 non-null      int64
 2   city    3 non-null      object
dtypes: int64(1), object(2)
memory usage: 204.0+ bytes
None
         age
count   3.0
mean   30.0
std     5.0
min    25.0
25%    27.5
50%    30.0
75%    32.5
max    35.0
      name  age    city
1      bob   30   paris
2  charlie   35  london
name       alice
age           25
city    new york
Name: 0, dtype: object
new vork
```

11249A338 exp 1.ipynb

File Edit View Insert Runtime Tools Help

Commands + Code + Text Run all

```
75%    32.5
max    35.0
      name  age    city
1      bob   30   paris
2  charlie   35  london
name       alice
age           25
city    new york
Name: 0, dtype: object
new york
name    0
age     0
city    0
dtype: int64
      name  age      city
0    alice   25  new york
1      bob   30     paris
2  charlie   35    london
      name  age      city
0    alice   25  new york
1      bob   30     paris
2  charlie   35    london
```

11249A338 exp 1.ipynb

File Edit View Insert Runtime Tools Help

Commands + Code + Text Run all

```
import pandas as pd
df=pd.read_csv("/content/student data.csv")
print(df)
print(df.head())
print(df[['age' , 'name']])
print(df[df['age']>13])
print(df.describe())
print(df.drop_duplicates())
```

```
   reg no     name  age dept section
0       1    alice   13  cse      s4
1       2      bob   19  cse      s4
2       3  charlie   18  cse      s4
3       4      bob   18  cse      s4
4       5  charlie   18  cse      s4
   reg no     name  age dept section
0       1    alice   13  cse      s4
1       2      bob   19  cse      s4
2       3  charlie   18  cse      s4
3       4      bob   18  cse      s4
4       5  charlie   18  cse      s4
   age     name
0   13    alice
1   19      bob
2   18  charlie
3   18      bob
4   18  charlie
   reg no     name  age dept section
1       2      bob   19  cse      s4
2       3  charlie   18  cse      s4
3       4      bob   18  cse      s4
```

11249A338 exp 1.ipynb

File Edit View Insert Runtime Tools Help

Commands + Code + Text Run all

```
   0       1    alice   13  cse      s4
   1       2      bob   19  cse      s4
   2       3  charlie   18  cse      s4
   3       4      bob   18  cse      s4
   4       5  charlie   18  cse      s4
      age     name
   0   13    alice
   1   19      bob
   2   18  charlie
   3   18      bob
   4   18  charlie
      reg no     name  age dept section
   1       2      bob   19  cse      s4
   2       3  charlie   18  cse      s4
   3       4      bob   18  cse      s4
   4       5  charlie   18  cse      s4
            reg no        age
   count  5.000000   5.000000
   mean   3.000000  17.200000
   std    1.581139   2.387467
   min    1.000000  13.000000
   25%    2.000000  18.000000
   50%    3.000000  18.000000
   75%    4.000000  18.000000
   max    5.000000  19.000000
      reg no     name  age dept section
   0       1    alice   13  cse      s4
   1       2      bob   19  cse      s4
   2       3  charlie   18  cse      s4
   3       4      bob   18  cse      s4
   4       5  charlie   18  cse      s4
```

MATPLOTLIB

11249A338 matplot-1.ipynb

File Edit View Insert Runtime Tools Help

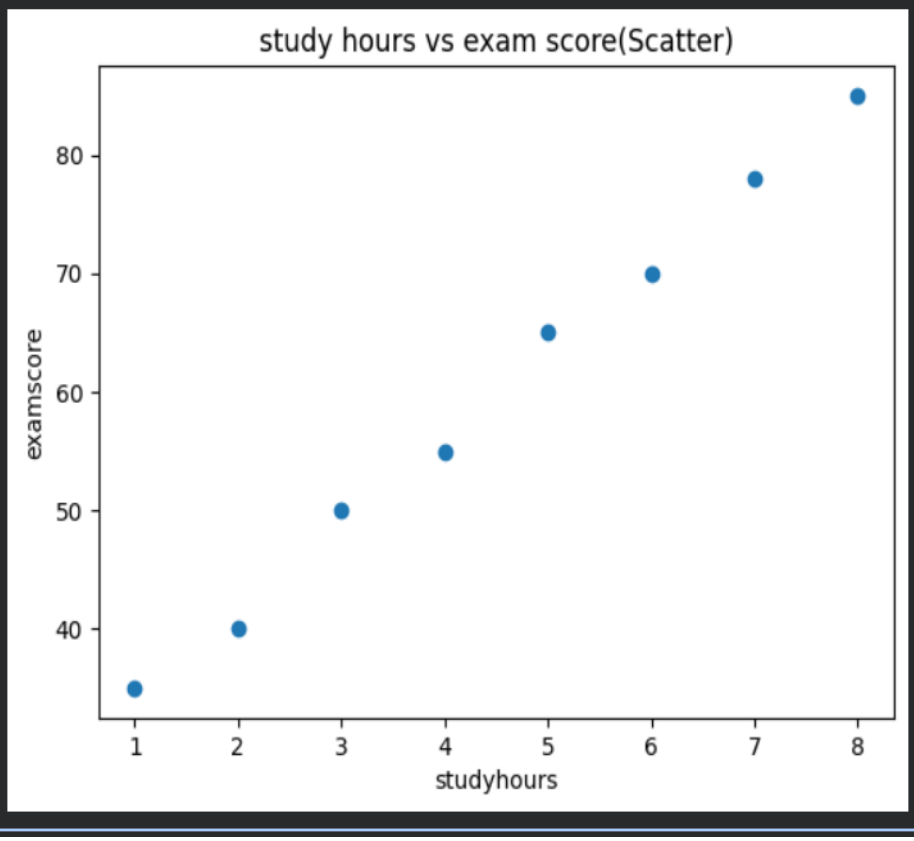
Commands + Code + Text Run all

```
import pandas as pd
import matplotlib.pyplot as plt
import numpy as np
data={
    'studyhours': [1,2,3,4,5,6,7,8],
    'examscore':[35,40,50,55,65,70,78,85]
}
df=pd.DataFrame(data)
plt.scatter(df['studyhours'],df['examscore'])
plt.xlabel("studyhours")
plt.ylabel("examscore")
plt.title("study hours vs exam score(Scatter)")
plt.show()
```

11249A338 matplot-1.ipynb

File Edit View Insert Runtime Tools Help

Commands + Code + Text Run all

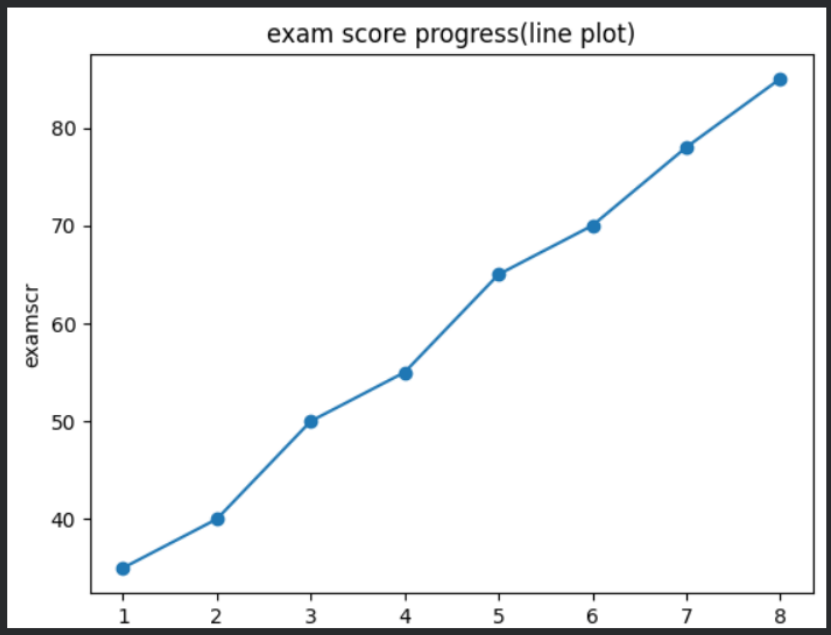


11249A338 matplot-1.ipynb

File Edit View Insert Runtime Tools Help

Commands + Code + Text Run all

```
plt.plot(df['studyhours'],df['examscore'],marker='o')
plt.xlabel("study hours")
plt.ylabel("examscr")
plt.title("exam score progress(line plot)")
plt.show()
```

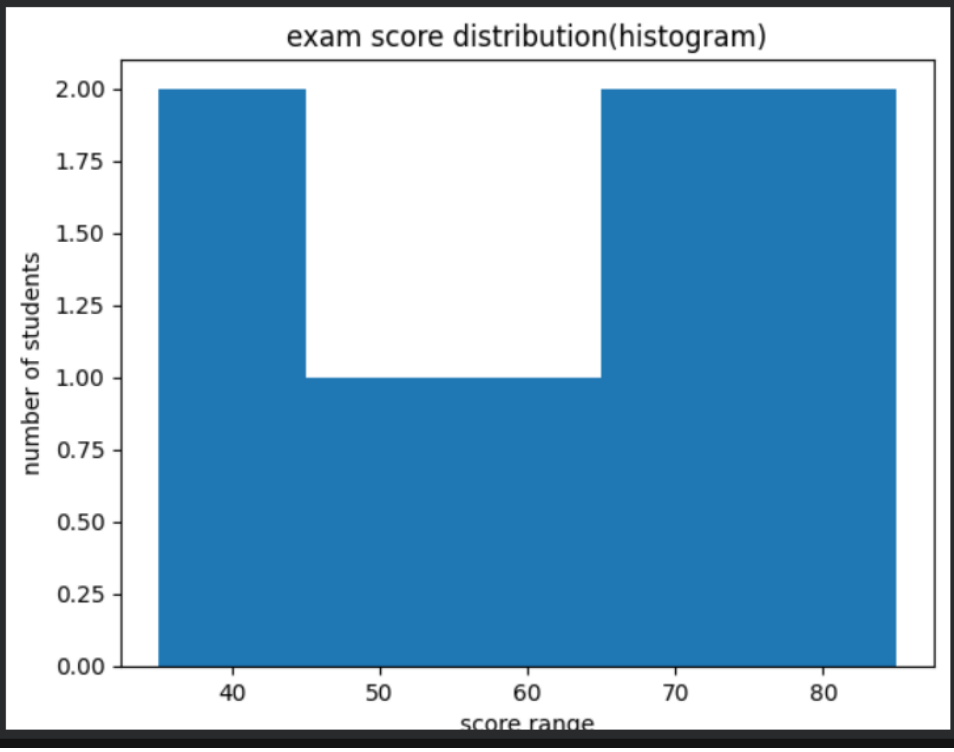


11249A338 matplot-1.ipynb

File Edit View Insert Runtime Tools Help

Commands + Code + Text Run all

```
plt.hist(df['examscore'],bins =5)
plt.xlabel("score range")
plt.ylabel("number of students")
plt.title("exam score distribution(histogram)")
plt.show()
```

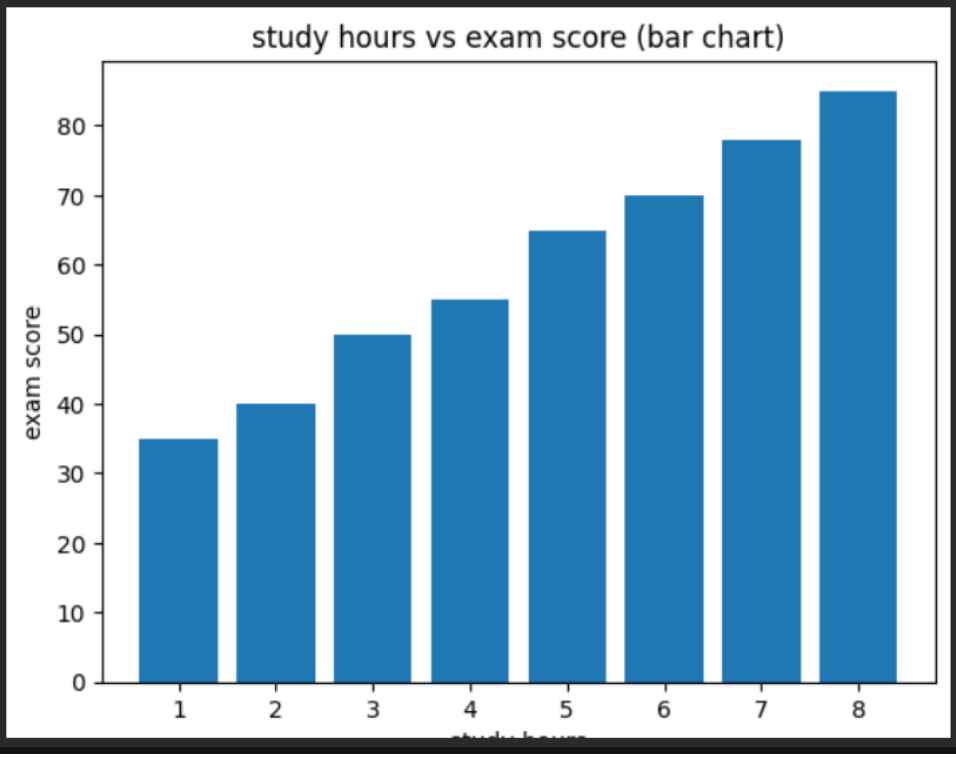


11249A338 matplot-1.ipynb

File Edit View Insert Runtime Tools Help

Commands + Code + Text Run all

```
plt.bar(df['studyhours'],df['examscore'])
plt.xlabel("study hours")
plt.ylabel("exam score")
plt.title("study hours vs exam score (bar chart)")
plt.show()
```



11249A338 matplot-1.ipynb

File Edit View Insert Runtime Tools Help

Commands + Code + Text Run all

```python
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_squared_error
data={
    'student':['s1','s2','s3','s4','s5','s6','s7','s8','s9','s10'],
    'classes_attended':[30,35,40,45,50,55,60,65,70,75],
    'internal_marks':[35,38,42,46,50,55,60,65,70,75]
}
df=pd.DataFrame(data)
print(df)
```

```
  student  classes_attended  internal_marks
0      s1                30              35
1      s2                35              38
2      s3                40              42
3      s4                45              46
4      s5                50              50
5      s6                55              55
6      s7                60              60
7      s8                65              65
8      s9                70              70
9     s10                75              75
```

11249A338 matplot-1.ipynb

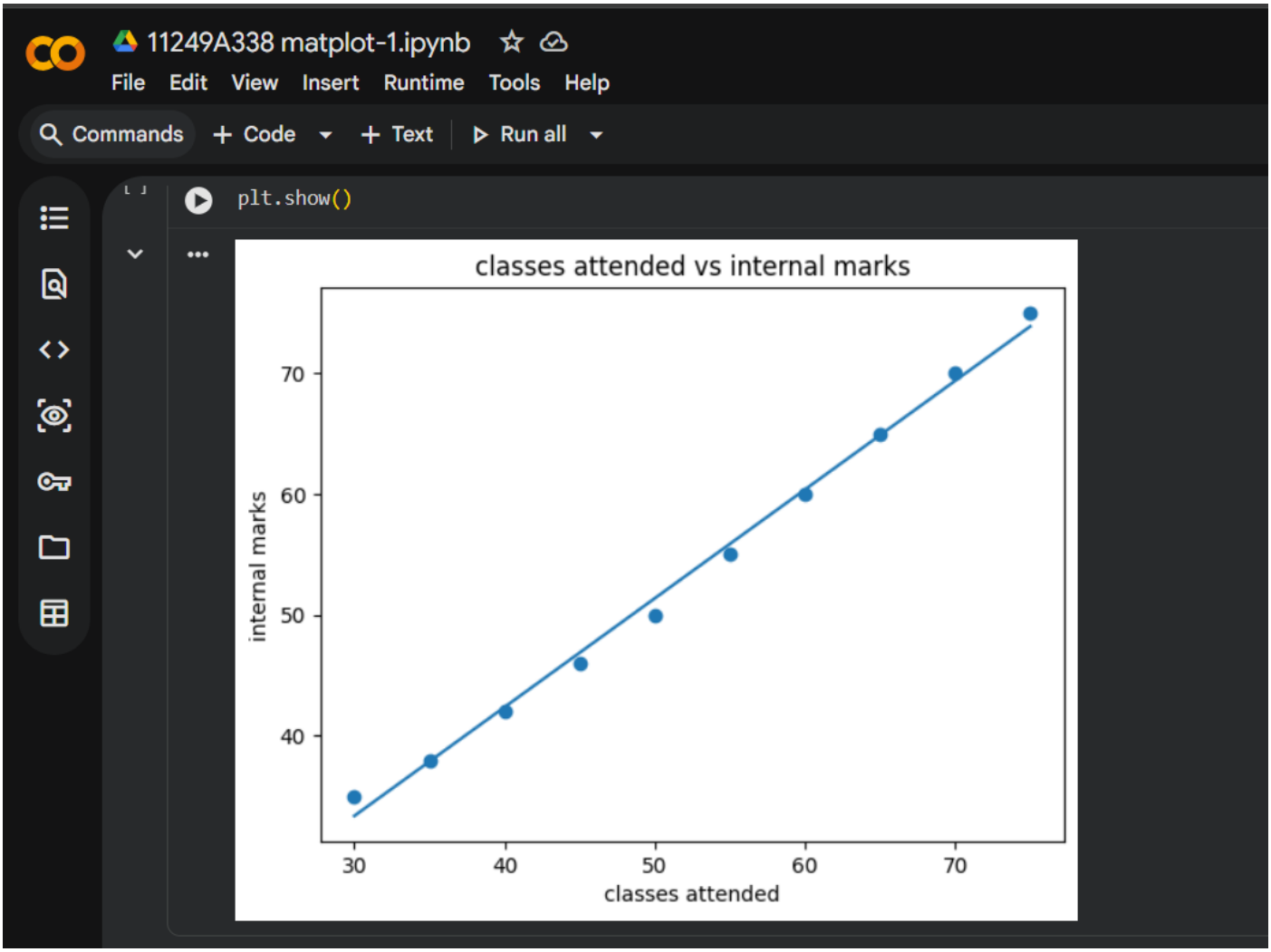
File Edit View Insert Runtime Tools Help

Commands + Code + Text Run all

```python
X=df[['classes_attended']]
Y=df['internal_marks']
X_train,X_test,Y_train,Y_test=train_test_split(
    X,Y,test_size=0.2,random_state=7
)
model=LinearRegression()
model.fit(X_train,Y_train)
predictions=model.predict(X_test)
mse=mean_squared_error(Y_test,predictions)
print("MSE:",mse)
print("Marks per class:",model.coef_[0])
print("Base Marks:",model.intercept_)
```

```
MSE: 0.578125
Marks per class: 0.9
Base Marks: 6.375
```

```python
plt.scatter(X,Y)
plt.plot(X,model.predict(X))
plt.xlabel("classes attended")
plt.ylabel("internal marks")
plt.title("classes attended vs internal marks")
plt.show()
```



RESULT: The different types of graphs was plotted successfully and understood the data visualization techniques.

DATA EXPLORATION AND PREPROCESSING IN PYTHON

EXP NO: 02
DATE: 14-02-2026

AIM: To perform data exploration and preprocessing using python libraries like numpy and pandas.

11249A338 exp 2.ipynb

File Edit View Insert Runtime Tools Help

Commands + Code + Text Run all

```python
import pandas as pd
import numpy as np
from sklearn.preprocessing import StandardScaler
from sklearn.model_selection import train_test_split
data={
    'country':['india','usa','india','usa','uk','india'],
    'age':[22,25,np.nan,30,28,35],
    'salary':[40000,60000,50000,np.nan,72000,58000],
    'purchased':['no','yes','yes','no','yes','yes']
}
df=pd.DataFrame(data)
print("ORIGINAL RAW DATA")
print(df)
print("\n")
print("MISSING VALUES")
print(df.isnull().sum())
print("\n")
df['age']=df['age'].fillna(df['age'].mean())
df['salary']=df['salary'].fillna(df['salary'].mean())
print("DATA AFTER CLEANING")
print(df)
print("\n")
df_encoded=pd.get_dummies(df,columns=['country'])
df_encoded['purchased']=df_encoded['purchased'].map({'yes':1,'no':0})
print("DATA AFTER ENCODING (all numbers now)")
print(df_encoded)
print("\n")
x=df_encoded.drop('purchased',axis=1)
```

11249A338 exp 2.ipynb

File Edit View Insert Runtime Tools Help

Commands + Code + Text Run all

```
print(df_encoded)
print("\n")
x=df_encoded.drop('purchased',axis=1)
y=df_encoded['purchased']
scaler=StandardScaler()
x_scaled=scaler.fit_transform(x)
print("FINAL PROCESSED DATA(ready for ai)")
print(pd.DataFrame(x_scaled,columns=x.columns).head())
```

```
ORIGINAL RAW DATA
  country   age   salary purchased
0   india  22.0  40000.0        no
1     usa  25.0  60000.0       yes
2   india   NaN  50000.0       yes
3     usa  30.0      NaN        no
4      uk  28.0  72000.0       yes
5   india  35.0  58000.0       yes


MISSING VALUES
country      0
age          1
salary       1
purchased    0
dtype: int64


DATA AFTER CLEANING
  country   age   salary purchased
0   india  22.0  40000.0        no
1     usa  25.0  60000.0       yes
```

11249A338 exp 2.ipynb

File Edit View Insert Runtime Tools Help

Commands + Code + Text Run all

```
salary       1
purchased    0
dtype: int64


DATA AFTER CLEANING
  country   age   salary purchased
0   india  22.0  40000.0        no
1     usa  25.0  60000.0       yes
2   india  28.0  50000.0       yes
3     usa  30.0  56000.0        no
4      uk  28.0  72000.0       yes
5   india  35.0  58000.0       yes


DATA AFTER ENCODING (all numbers now)
    age   salary  purchased  country_india  country_uk  country_usa
0  22.0  40000.0          0           True       False        False
1  25.0  60000.0          1          False       False         True
2  28.0  50000.0          1           True       False        False
3  30.0  56000.0          0          False       False         True
4  28.0  72000.0          1          False        True        False
5  35.0  58000.0          1           True       False        False


FINAL PROCESSED DATA(ready for ai)
        age    salary  country_india  country_uk  country_usa
0 -1.484615 -1.644453            1.0   -0.447214    -0.707107
1 -0.742307  0.411113           -1.0   -0.447214     1.414214
2  0.000000 -0.616670            1.0   -0.447214    -0.707107
3  0.494872  0.000000           -1.0   -0.447214     1.414214
4  0.000000  1.644453           -1.0    2.236068    -0.707107
```

RESULT: The data was successfully cleaned, transformed and analyzed the dataset by handling missing values and preparing it for modelling.

IMPLEMENT REGULARIZED LINEAR REGRESSION

EXP NO: 03
DATE: 14-02-2026

AIM: To implement regularization to reduce overfitting and to improve model performance.

11249A338 exp 3.ipynb

File Edit View Insert Runtime Tools Help

Commands + Code + Text Run all

[3]

```python
from sklearn.linear_model import LinearRegression, Ridge, Lasso
from sklearn.metrics import mean_squared_error
from sklearn import datasets
from sklearn.model_selection import train_test_split

diabetes=datasets.load_diabetes()
x_train, x_test, y_train, y_test = train_test_split(diabetes.data,diabetes.target, test_size=0.2, random_state=42)
lr=LinearRegression().fit(x_train,y_train)
ridge=Ridge(alpha=1.0).fit(x_train,y_train)
lasso=Lasso(alpha=0.1).fit(x_train,y_train)
print(f"linear regresion MSE:{mean_squared_error(y_test,lr.predict(x_test)):.2f}")
print(f"ridge regression MSE:{mean_squared_error(y_test,ridge.predict(x_test)):.2f}")
print(f"lasso regression MSE:{mean_squared_error(y_test,lasso.predict(x_test)):.2f}")
```

```
linear regresion MSE:2900.19
ridge regression MSE:3077.42
lasso regression MSE:2798.19
```

RESULT: The regularization was applied successfully to reduce overfitting and to improve model performance.

IMPLEMENT NAÏVE BAYES CLASSIFICATION

EXP NO: 04
DATE: 23-02-2026

AIM: To implement Naïve Bayes Classification for predicting class labels.

```python
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.naive_bayes import GaussianNB
from sklearn.metrics import accuracy_score
data={
    'studyhours' :[5,2,8,1,6,3,7,4],
    'attendance':[80,50,90,40,85,60,88,65],
    'result':['pass','fail','pass','fail','pass','fail','pass','fail']
}
df=pd.DataFrame(data)
x=df[['studyhours','attendance']]
y=df['result']
x_train, x_test, y_train, y_test=train_test_split( *arrays: x,y,test_size=0.25, random_state=42)
model =GaussianNB()
model.fit(x_train,y_train)
y_pred=model.predict(x_test)
accuracy=accuracy_score(y_test,y_pred)
print("Predicted  Results:",y_pred)
print("Actual Results:", y_test.values)
print("Accuracy :",accuracy)
```

```
C:\Users\scsvm\PycharmProjects\11249A338\11249A338\.venv\Scripts\python.exe "C:\Users\scsvm\PycharmProjects\11249A338\11249A338\exp 4.py"
Predicted  Results: ['fail' 'fail']
Actual Results: ['fail' 'fail']
Accuracy : 1.0
```

RESULT: Successfully built a probabilistic classifier and achieved accurate predictions on the dataset.

IMPLEMENT REGULARIZED LOGISTIC REGRESSION

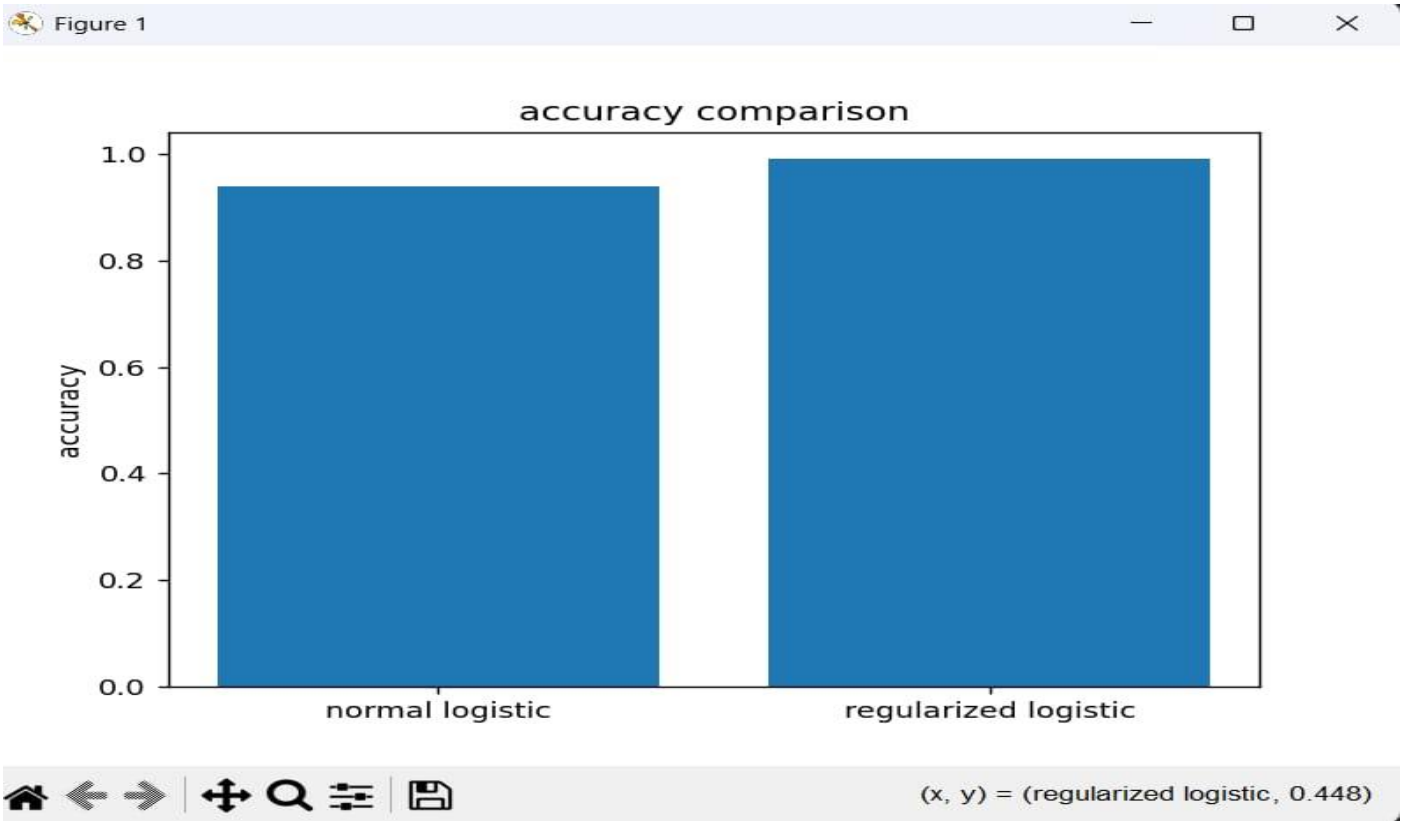
EXP NO: 05
DATE: 02-03-2026

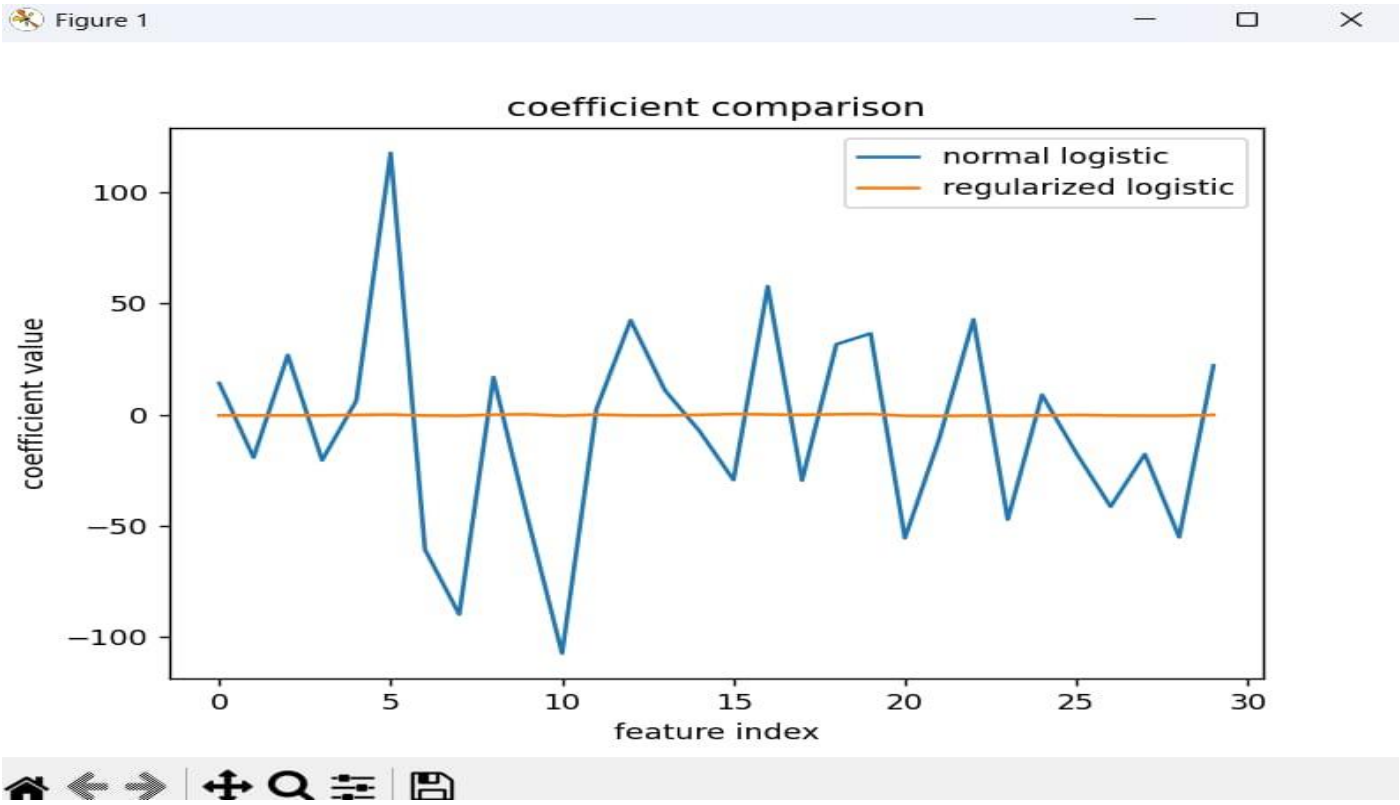
AIM: To implement Regularized Logistic Regression using python and analyze the effect of regularization in reducing overfitting and improving model performance.

```python
import numpy as np
import matplotlib.pyplot as plt
from sklearn.datasets import load_breast_cancer
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score
data=load_breast_cancer()
x=data.data
y=data.target
x_train,x_test,y_train,y_test= train_test_split( *arrays: x,y,test_size=0.2,random_state=42)
scaler=StandardScaler()
x_train_scaled = scaler.fit_transform(x_train)
x_test_scaled=scaler.transform( x_test)
normal_model=LogisticRegression(C=1e6,penalty='l2',solver='liblinear')
normal_model.fit(x_train_scaled , y_train)
normal_pred=normal_model.predict(x_test_scaled)
normal_acc=accuracy_score(y_test,normal_pred)
regularized_model=LogisticRegression(C=0.1,penalty='l2',solver='liblinear')
regularized_model.fit(x_train_scaled,y_train)
reg_pred=regularized_model.predict(x_test_scaled)
reg_acc=accuracy_score(y_test,reg_pred)
print("normal logistic accuracy:",normal_acc*100)
print("regularized logistic accuracy:",reg_acc*100)
plt.figure()
plt.plot( *args: normal_model.coef_.flatten(),label='normal logistic')
```

```python
regularized_model.fit(x_train_scaled,y_train)
reg_pred=regularized_model.predict(x_test_scaled)
reg_acc=accuracy_score(y_test,reg_pred)
print("normal logistic accuracy:",normal_acc*100)
print("regularized logistic accuracy:",reg_acc*100)
plt.figure()
plt.plot( *args: normal_model.coef_.flatten(),label='normal logistic')
plt.plot( *args: regularized_model.coef_.flatten(),label='regularized logistic')
plt.legend()
plt.title("coefficient comparison")
plt.xlabel("feature index")
plt.ylabel("coefficient value")
plt.show()
plt.figure()
models=['normal logistic','regularized logistic']
accuracies=[normal_acc,reg_acc]
plt.bar(models,accuracies)
plt.title("accuracy comparison")
plt.ylabel("accuracy")
plt.show()
```

```
C:\Users\scsvm\PycharmProjects\11249A338\11249A338\.venv\Scripts\python.exe "C:\Users\scsvm\PycharmProjects\11249A338\11249A338\exp 5.py"
normal logistic accuracy: 93.85964912280701
regularized logistic accuracy: 99.12280701754386
```





RESULT: Successfully implemented logistic regression and observed the regularization improves generalization by controlling model complexity, leading to better prediction.

BUILD MODELS USING DIFFERENT ENSEMBLING TECHNIQUES

EXP NO: 06
DATE: 09-03-2026

AIM: To build models using different ensemble techniques like Boosting, Random Forest.

```python
from sklearn import datasets
from sklearn.ensemble import RandomForestClassifier,AdaBoostClassifier,StackingClassifier
from sklearn.svm import SVC
from sklearn.linear_model import LogisticRegression
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score
iris=datasets.load_iris()
x_train,x_test,y_train,y_test=train_test_split( *arrays: iris.data,iris.target,test_size=0.2,random_state=42)
rf=RandomForestClassifier(n_estimators=100).fit(x_train,y_train)
ada=AdaBoostClassifier(n_estimators=100).fit(x_train,y_train)
estimators=[('rf',RandomForestClassifier()),('svc',SVC(probability=True))]
stack=StackingClassifier(estimators=estimators,final_estimator=LogisticRegression()).fit(x_train,y_train)
print(f"Random Forest Accuracy: {accuracy_score(y_test, rf.predict(x_test)): .2f}")
print(f"AdaBoost Accuracy : {accuracy_score(y_test,ada.predict(x_test)):.2f}")
print(f"Stacking Accuracy :{ accuracy_score(y_test,stack.predict(x_test)):.2f}")
```

```
C:\Users\scsvm\PycharmProjects\11249A338\11249A338\.venv\Scripts\python.exe "C:\Users\scsvm\PycharmProjects\11249A338\11249A338\exp 6.py"
Random Forest Accuracy:  1.00
AdaBoost Accuracy : 0.93
Stacking Accuracy :1.00

Process finished with exit code 0
```

RESULT: Successfully improved prediction accuracy by combining multiple models.

BUILD MODELS USING DECISION TREES

EXP NO: 07
DATE: 16-03-2026

AIM: To build and analyze models using Decision Tree algorithms.

```
from sklearn import datasets
from sklearn.tree import DecisionTreeClassifier
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score
iris=datasets.load_iris()
x_train,x_test,y_train,y_test=train_test_split(*arrays: iris.data,iris.target,test_size=0.2,random_state=42)
dt_clf=DecisionTreeClassifier(max_depth=3).fit(x_train,y_train)
print(f"Decision Tree Classifier Accuracy :{accuracy_score(y_test,dt_clf.predict(x_test)):.2f}")
print("Feature Importances :",dt_clf.feature_importances_)
```

```
C:\Users\scsvm\PycharmProjects\11249A338\11249A338\.venv\Scripts\python.exe "C:\Users\scsvm\PycharmProjects\11249A338\11249A338\exp 7.py"
Decision Tree Classifier Accuracy :1.00
Feature Importances : [0.        0.        0.93462632 0.06537368]

Process finished with exit code 0
```

RESULT: Successfully created decision tree models and interpreted decision rules for classification.

SUPPORT VECTOR MACHINES (SVM) WITH DIFFERENT KERNELS

EXP NO: 08
DATE: 16-03-2026

AIM: To implement SVM with different kernels (linear,polynomial,RBF).

```python
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.svm import SVC
from sklearn.metrics import accuracy_score
from sklearn.datasets import load_iris
data=load_iris()
x=data.data
y=data.target
x_train,x_test,y_train,y_test=train_test_split( *arrays: x,y,test_size=0.2)
model_linear=SVC(kernel='linear')
model_linear.fit(x_train,y_train)
pred_linear=model_linear.predict(x_test)
acc_linear=accuracy_score(y_test,pred_linear)
print("Linear Kernel Accuracy:",acc_linear)
model_poly=SVC(kernel='poly',degree=3)
model_poly.fit(x_train,y_train)
pred_poly=model_poly.predict(x_test)
acc_poly=accuracy_score(y_test,pred_poly)
print("Polynomial Kernel Accuracy:",acc_poly)
model_rbf=SVC(kernel='rbf')
model_rbf.fit(x_train,y_train)
pred_rbf=model_rbf.predict(x_test)
acc_rbf=accuracy_score(y_test,pred_rbf)
print("RBF Kernel Accuracy:",acc_rbf)
```


```python
x_train,x_test,y_train,y_test=train_test_split( *arrays: x,y,test_size=0.2)
model_linear=SVC(kernel='linear')
model_linear.fit(x_train,y_train)
pred_linear=model_linear.predict(x_test)
acc_linear=accuracy_score(y_test,pred_linear)
print("Linear Kernel Accuracy:",acc_linear)
model_poly=SVC(kernel='poly',degree=3)
model_poly.fit(x_train,y_train)
pred_poly=model_poly.predict(x_test)
acc_poly=accuracy_score(y_test,pred_poly)
print("Polynomial Kernel Accuracy:",acc_poly)
model_rbf=SVC(kernel='rbf')
model_rbf.fit(x_train,y_train)
pred_rbf=model_rbf.predict(x_test)
acc_rbf=accuracy_score(y_test,pred_rbf)
print("RBF Kernel Accuracy:",acc_rbf)
```


```
C:\Users\scsvm\PycharmProjects\11249A338\11249A338\.venv\Scripts\python.exe "C:\Users\scsvm\PycharmProjects\11249A338\11249A338\exp 8.py"
Linear Kernel Accuracy: 1.0
Polynomial Kernel Accuracy: 1.0
RBF Kernel Accuracy: 0.9666666666666667

Process finished with exit code 0
```

RESULT: Successfully classified data using SVM and compared performance.

IMPLEMENT KNN CLASSIFICATION

EXP NO: 09

DATE: 23-03-2026

AIM: To implement KNN classification algorithms.

```python
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.neighbors import KNeighborsClassifier
from sklearn.metrics import accuracy_score, confusion_matrix, classification_report
data = pd.read_csv("mobile_price.csv")
X = data.drop(labels: "price_range", axis=1)
y = data["price_range"]
X_train, X_test, y_train, y_test = train_test_split(
    *arrays: X, y, test_size=0.2, random_state=42)
scaler = StandardScaler()
X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)
knn = KNeighborsClassifier(n_neighbors=5)
knn.fit(X_train, y_train)
y_pred = knn.predict(X_test)
print("Accuracy:", accuracy_score(y_test, y_pred))
print("\nConfusion Matrix:\n", confusion_matrix(y_test, y_pred))
print("\nClassification Report:\n", classification_report(y_test, y_pred))
```

```
C:\Users\scsvm\PycharmProjects\11249A338\11249A338\.venv\Scripts\python.exe "C:\Users\scsvm\PycharmProjects\11249A338\11249A338\exp 9.py"
Accuracy: 0.53

Confusion Matrix:
 [[76 24  5  0]
```

```
C:\Users\scsvm\PycharmProjects\11249A338\11249A338\.venv\Scripts\python.exe "C:\Users\scsvm\PycharmProjects\11249A338\11249A338\exp 9.py"
Accuracy: 0.53

Confusion Matrix:
 [[76 24  5  0]
 [25 41 22  3]
 [ 6 41 34 11]
 [ 2 13 36 61]]

Classification Report:
              precision    recall  f1-score   support

           0       0.70      0.72      0.71       105
           1       0.34      0.45      0.39        91
           2       0.35      0.37      0.36        92
           3       0.81      0.54      0.65       112

    accuracy                           0.53       400
   macro avg       0.55      0.52      0.53       400
weighted avg       0.57      0.53      0.54       400


Process finished with exit code 0
```

RESULT: Successfully classified data based on nearest neighbours.

K-MEANS CLUSTERING WITH PCA AND ELBOW METHOD

EXP NO: 10
DATE: 23-03-2026

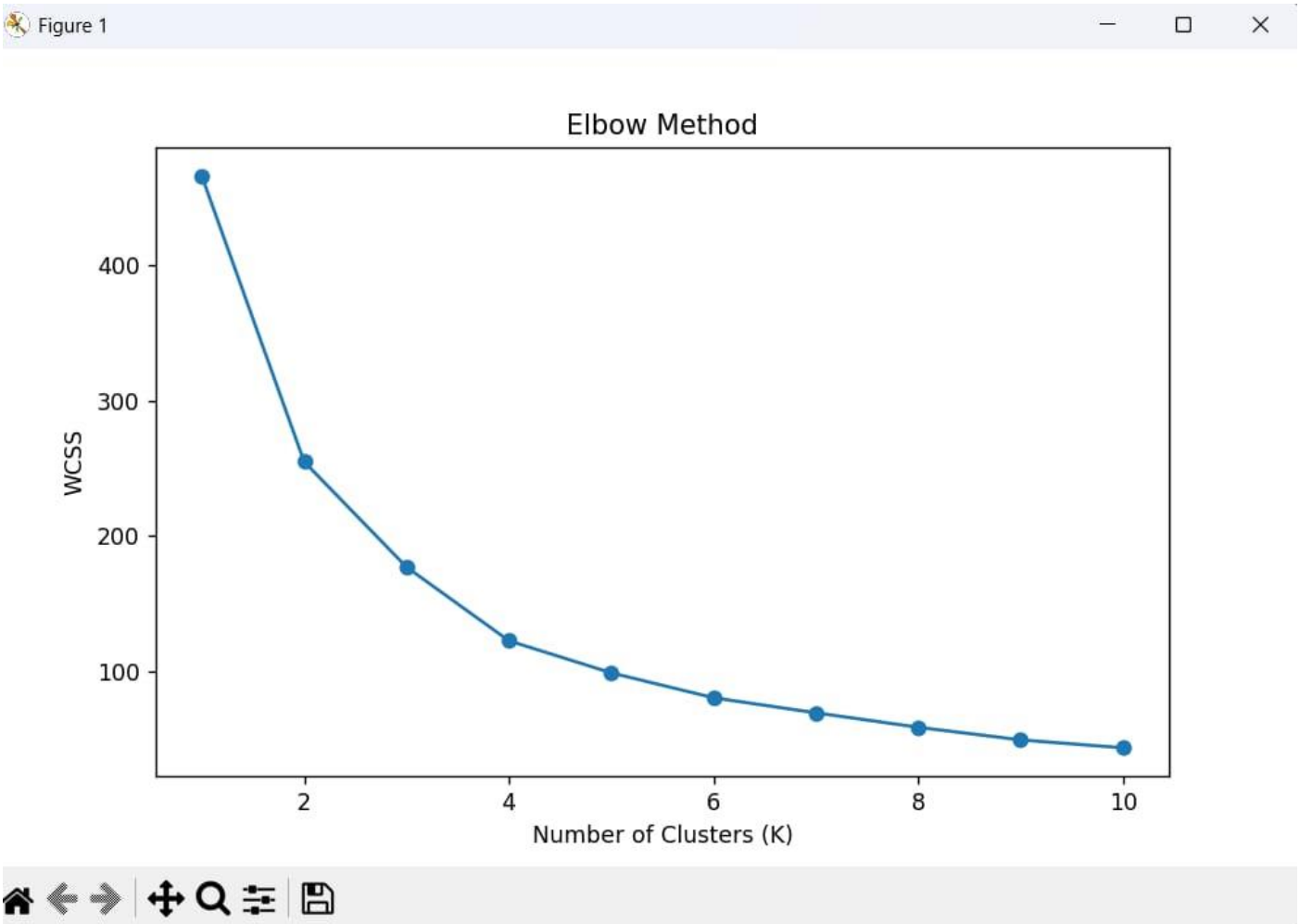
AIM: To perform K-Means clustering and determine optimal clusters using PCA and Elbow method.

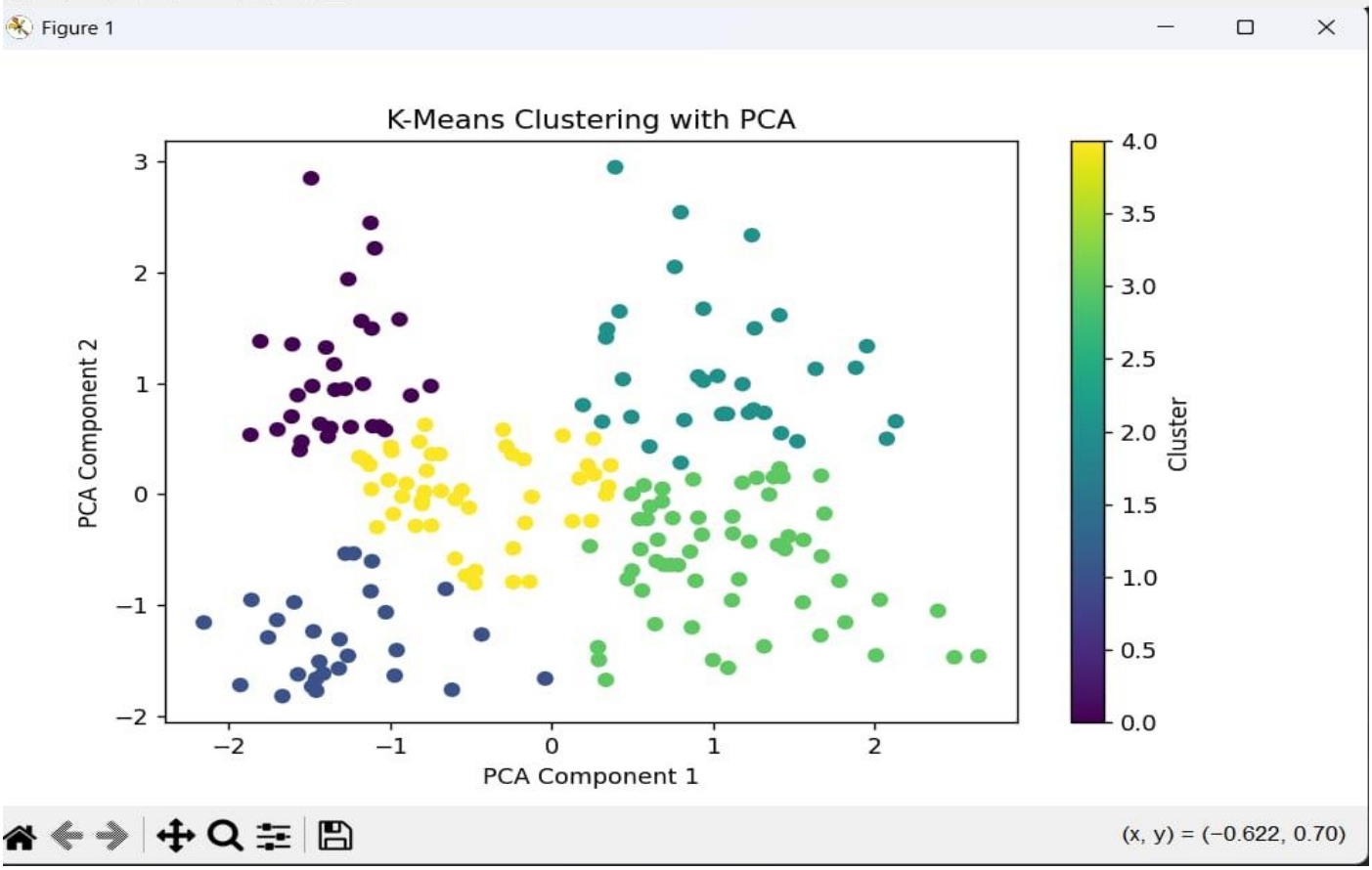
```python
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.preprocessing import StandardScaler
from sklearn.decomposition import PCA
from sklearn.cluster import KMeans
data = pd.read_csv("Mall_Customers.csv")
X = data[['Age', 'Annual Income (k$)', 'Spending Score (1-100)']]
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)
pca = PCA(n_components=2)
X_pca = pca.fit_transform(X_scaled)
print("Explained Variance Ratio:", pca.explained_variance_ratio_)
wcss = []
for i in range(1, 11):
    kmeans = KMeans(n_clusters=i, random_state=0, n_init=10)
    kmeans.fit(X_pca)
    wcss.append(kmeans.inertia_)
plt.figure(figsize=(8, 5))
plt.plot( *args: range(1, 11), wcss, marker='o')
plt.title("Elbow Method")
plt.xlabel("Number of Clusters (K)")
plt.ylabel("WCSS")
plt.show()
kmeans = KMeans(n_clusters=5, random_state=0, n_init=10)
clusters = kmeans.fit_predict(X_pca)
plt.figure(figsize=(8, 5))
```

```python
    kmeans.fit(X_pca)
    wcss.append(kmeans.inertia_)
plt.figure(figsize=(8, 5))
plt.plot( *args: range(1, 11), wcss, marker='o')
plt.title("Elbow Method")
plt.xlabel("Number of Clusters (K)")
plt.ylabel("WCSS")
plt.show()
kmeans = KMeans(n_clusters=5, random_state=0, n_init=10)
clusters = kmeans.fit_predict(X_pca)
plt.figure(figsize=(8, 5))
plt.scatter(X_pca[:, 0], X_pca[:, 1], c=clusters, cmap='viridis')
plt.title("K-Means Clustering with PCA")
plt.xlabel("PCA Component 1")
plt.ylabel("PCA Component 2")
plt.colorbar(label='Cluster')
plt.show()
```

```
C:\Users\scsvm\PycharmProjects\11249A338\11249A338\.venv\Scripts\python.exe "C:\Users\scsvm\PycharmProjects\11249A338\11249A338\exp 10.py"
Explained Variance Ratio: [0.44266167 0.33308378]

Process finished with exit code 0
```





RESULT: Successfully reduced dimensionally using PCA and identified optimal clusters using the Elbow method.